Weather Finder Pro

PYTHON SEMESTER PROJECT & APPLICATION DEMO

Built with Python, Tkinter, and OpenWeather API

Core Logic: WeatherService

The Foundation Class

The **WeatherService** class handles the "heavy lifting" of the application:

- Encapsulates API Key and Base URL.
- Uses the requests library to handle HTTP GET calls.
- Processes the JSON responses into usable Python dictionaries.

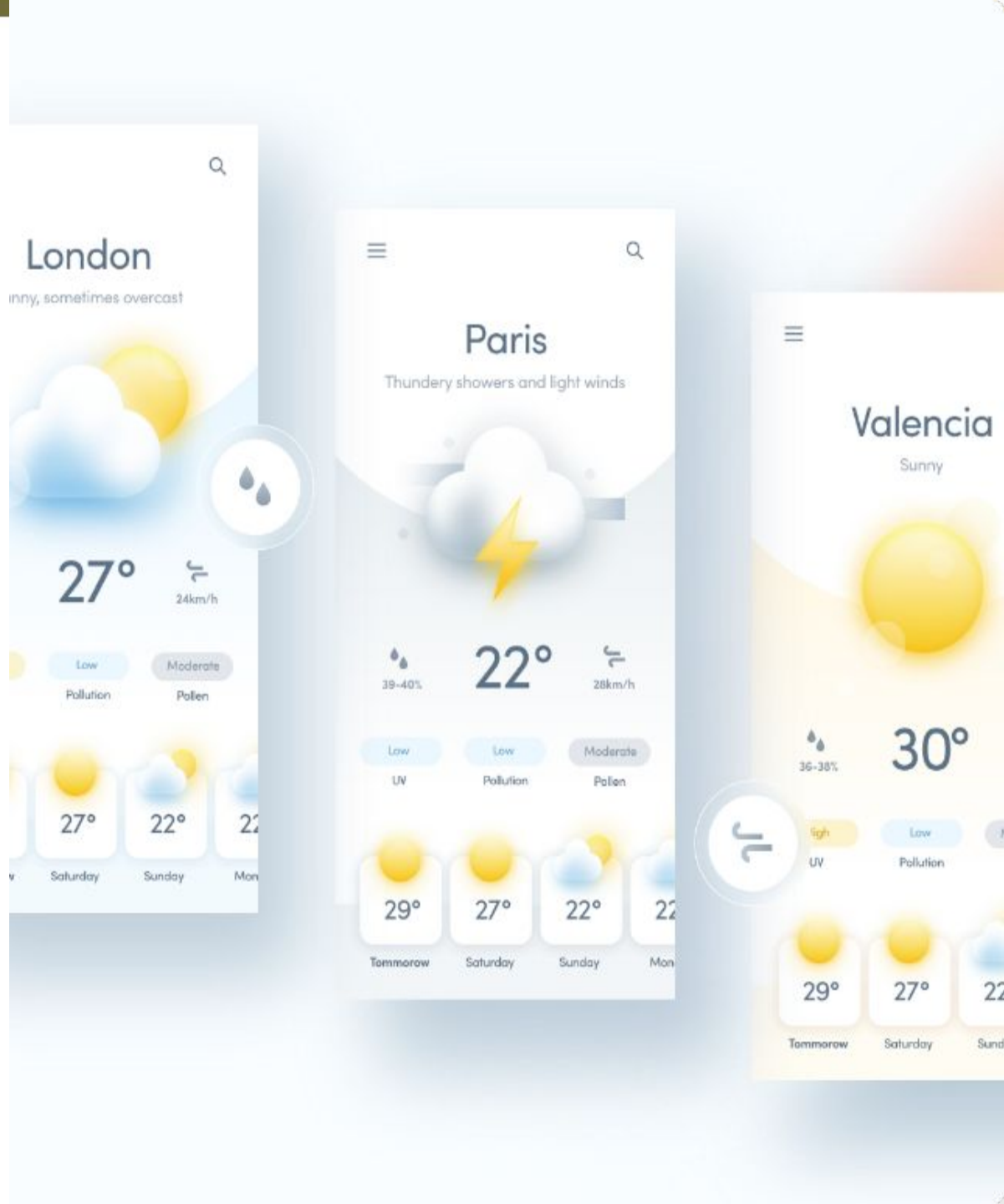
```python
class WeatherService:
    def __init__(self, api_key):
        self.api_key = api_key
        self.base_url = "http://api.openweathermap.org/data/2.5/weather"

    def get_data(self, city):
        #params for the request, units=metric gives us celsius
        #if we want fahrenheit we would change metric to imperial
        params = {
            'q': city,
            'appid': self.api_key,
            'units': 'metric'
        }
        response = requests.get(self.base_url, params=params)
        return response.json()
```

Advanced OOP: Inheritance

Extending Functionality

```python
#subclass for the actual app functionality
#this inherits from the service class to satisfy the project specs
class WeatherApp(WeatherService):
    def __init__(self, root, api_key):
        #calling the parent constructor
        super().__init__(api_key)
        self.root = root
        self.root.title("Weather Finder")
        self.root.geometry("400x300")

        #setting up the gui components
        self.label = tk.Label(root, text="enter city name:", font=("Arial", 12))
        self.label.pack(pady=10)

        self.city_entry = tk.Entry(root, font=("Arial", 12))
        self.city_entry.pack(pady=5)

        self.search_btn = tk.Button(root, text="get weather", command=self.display_weather)
        self.search_btn.pack(pady=10)

        self.result_label = tk.Label(root, text="", font=("Arial", 11), justify="left")
        self.result_label.pack(pady=20)
```

The WeatherApp subclass inherits all API logic while adding GUI elements and user interaction handling.



Data Mapping

The application parses a JSON payload to extract key metrics for the user:

Metric	JSON Key Path	User Benefit
Temperature	['main']['temp']	Current heat in Celsius/Fahrenheit
Conditions	['weather'][0]['description']	Visual sky report (Clear, Clouds, etc.)
Humidity	['main']['humidity']	Relative moisture percentage
"Feels Like"	['main']['feels_like']	Adjusted temp for wind/humidity

Project Demo

GitHub Repo: https://github.com/atwhite2006/Python_Weather_Project